The end-to-end project

California housing

LECTURE 2 | GÉRON CH. 2

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Last lecture we framed the brief and looked at the whole dataset once. Everything you need from that, in one table:

What we found	Why it matters today
The output is consumed as a *price*	supervised regression: one number per district, error in dollars
207 missing values in `total_bedrooms`; `ocean_proximity` is categorical, one level with n=5	both must be handled before anything is fitted
A target capped: 965 districts at the cap, $500,001	4.7% of districts carry a label that is not the answer
Scales four orders of magnitude apart, many columns skewed right	matters for some methods; not for the least squares we derive today
Nothing split yet	so the split comes first, before we look any closer

✗ We stopped at exactly the moment to stop: four histograms in, before plotting one variable against another. Today starts there.

Today

1. **The split** — why the test set is set aside before any exploration, and a stratified split, measured against a random one
2. **Exploring the training set** — geography, a colour scale that lies, correlations and what they miss, ratios worth more than their columns
3. **The mathematics** — least squares: what fitting a linear model minimises, and why the point where its gradient vanishes is the minimum
4. **Design exercise** — a rental marketplace's suggested listing price, in nine steps, with the pitfalls and how to address them

By the end you will have a sealed test set, a training set you have looked at properly, and the objective a linear model minimises — with a proof that it has no false minima.

PART ONE

The split

set the test set aside, then measure the split

Set it aside *now*, and do not look

DATA SNOOPING BIAS

Your brain is an outstanding pattern detector, which means it overfits. If you study the test set, you will pick a model that suits it, your estimate of generalisation error will be optimistic, and you will ship something that underperforms.

✗ Create the test set before you explore any further.

The obvious split, and its flaw

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing_full, test_size=0.2,
                                       random_state=42)
```

Purely random sampling is fine *if the dataset is large enough relative to the number of attributes*.

✗ Ours may not be. Random sampling risks significant sampling bias.

Stratified sampling

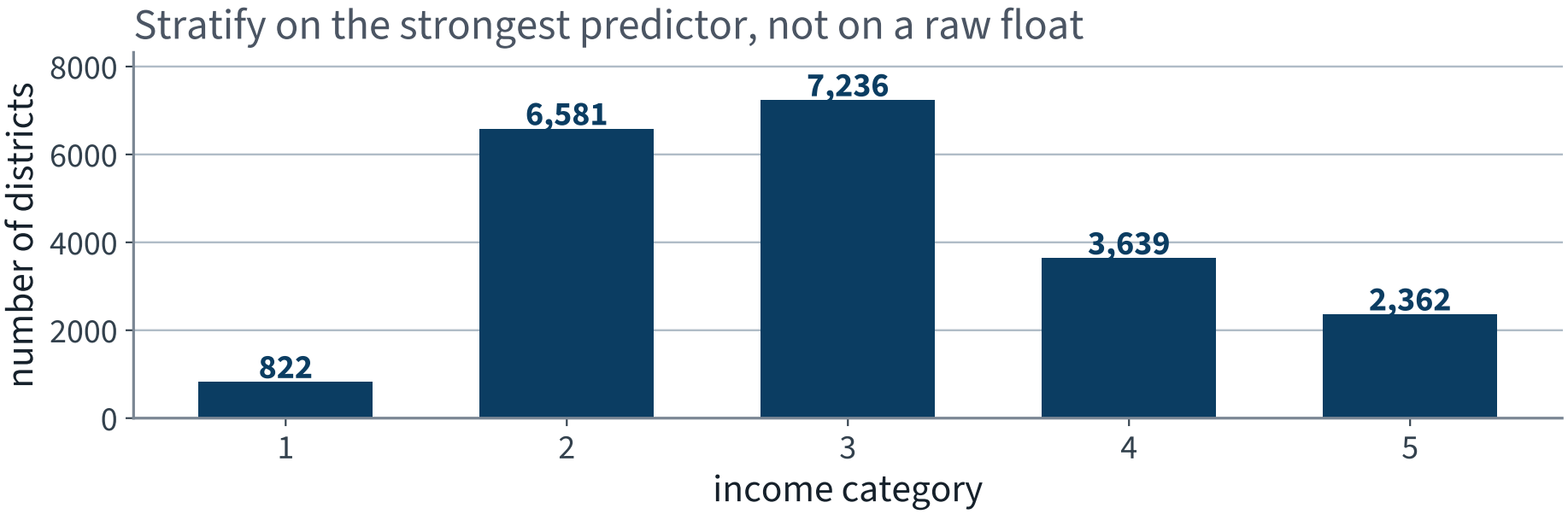
Surveyors do not call 1,000 people at random. They ensure the sample is representative of the population on the dimensions that matter.

Experts tell us **median income** predicts price. So our test set must be representative across income levels.

```
import numpy as np

housing_full["income_cat"] = pd.cut(housing_full["median_income"],
                                    bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                                    labels=[1, 2, 3, 4, 5])
```

Five strata, none of them tiny



The bins are chosen so that no stratum is too small to sample from. Category 1 has 822 districts — enough.

Split on the strata

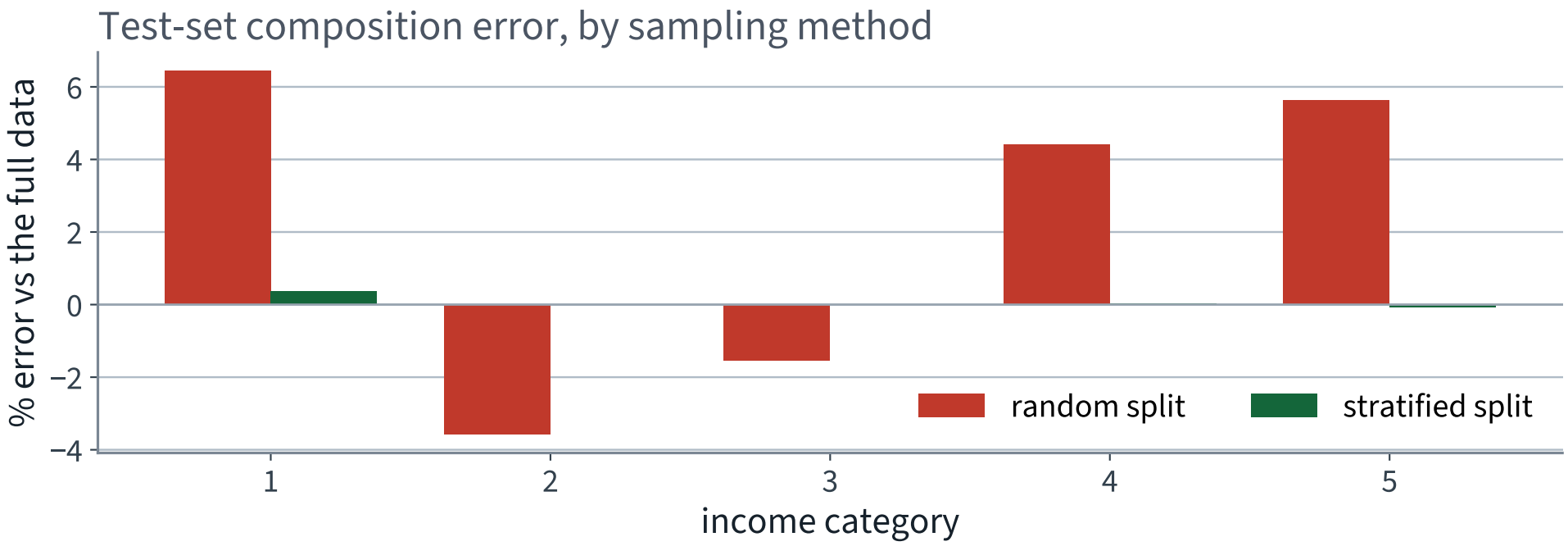
```
strat_train_set, strat_test_set = train_test_split(
    housing_full, test_size=0.2,
    stratify=housing_full["income_cat"], random_state=42)

for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

The stratified test set has income proportions almost identical to the full dataset. The purely random one is visibly skewed.

Not too many strata, and each stratum large enough — or the estimate of a stratum's importance is itself biased.

Does it actually matter? Measure it.



✗ Random split: up to 6.4% off. ✓ Stratified: up to 0.36% off. Roughly eighteen times better, for one keyword argument.

Check the split before you trust it

```
assert len(strat_train_set) + len(strat_test_set) == 20640
assert set(strat_train_set.columns) == set(strat_test_set.columns)
assert strat_train_set.index.intersection(strat_test_set.index).empty
```

Three lines: nothing was dropped, the two frames have the same columns, and no district is in both. Each one has been a real bug in a real notebook.

> **ASSERT AFTER EVERY STRUCTURAL STEP**
>
> An assertion is a claim about your data that fails *loudly*. It is the only thing in this course that does. Write one after every split, merge, reshape and drop — and count what you dropped.

PART TWO

Exploring the training set

look properly — on the training set, and only that

Now explore — the training set, and only that

```
housing = strat_train_set.copy()          # 16,512 districts. The other 4,128
                                          # are sealed until the final
                                          # evaluation.
```

Everything from here to the end of this part is computed on `housing`. Every correlation, every scatter, every ratio.

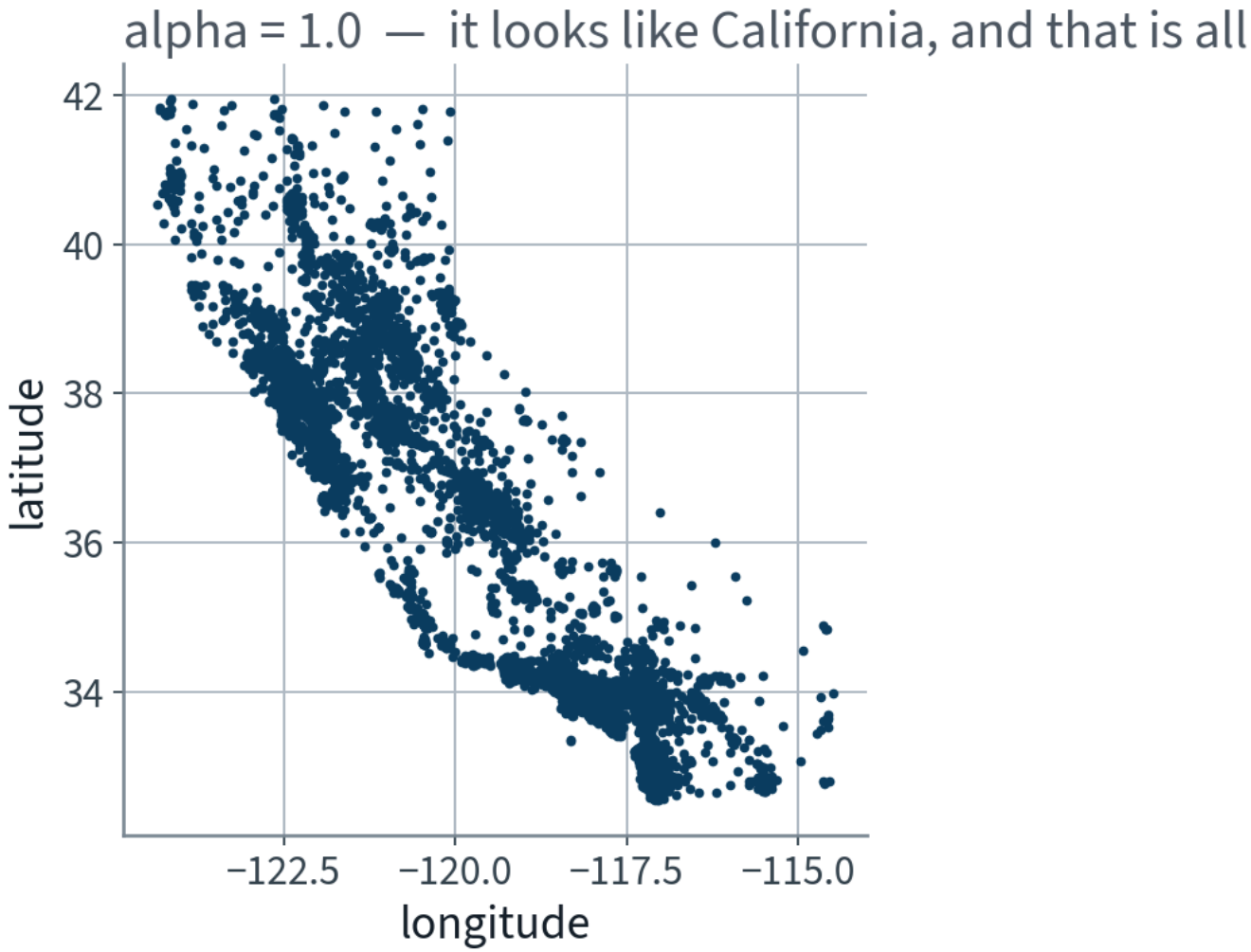
> **WHY THE COPY**
>
> So that nothing we invent while exploring — a new column, a filter, a dropped row — can reach `strat_train_set` by accident. And so that the numbers on the next eight slides are honestly labelled: they are **training-set** numbers, and they will differ slightly from anything you compute on all 20,640 rows.

The data is geographical — so look at it

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
plt.show()
```

It looks like California. Beyond that, nothing is visible.

Every district, plotted



California. And nothing else — the ink is saturated.

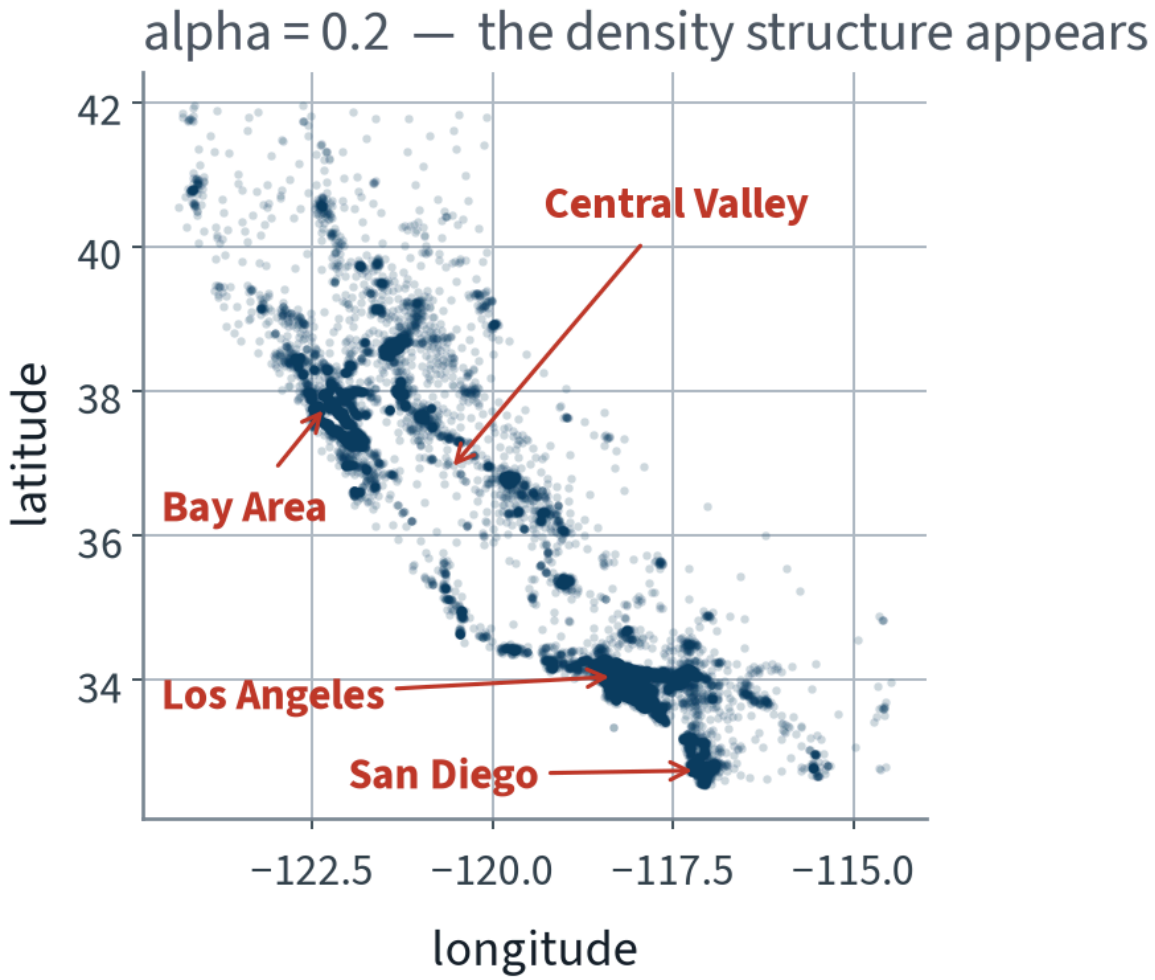
One parameter changes everything

```
housing.plot(kind="scatter", x="longitude", y="latitude",
             grid=True, alpha=0.2)
plt.show()
```

Now the high-density areas appear: the Bay Area, Los Angeles, San Diego, and the Central Valley around Sacramento and Fresno.

Our brains are excellent at spotting patterns in pictures. Expect to tune plotting parameters until the pattern shows.

The same data, at one fifth the opacity



One keyword argument. Same data, and now you can see the population of a state.

Three variables at once

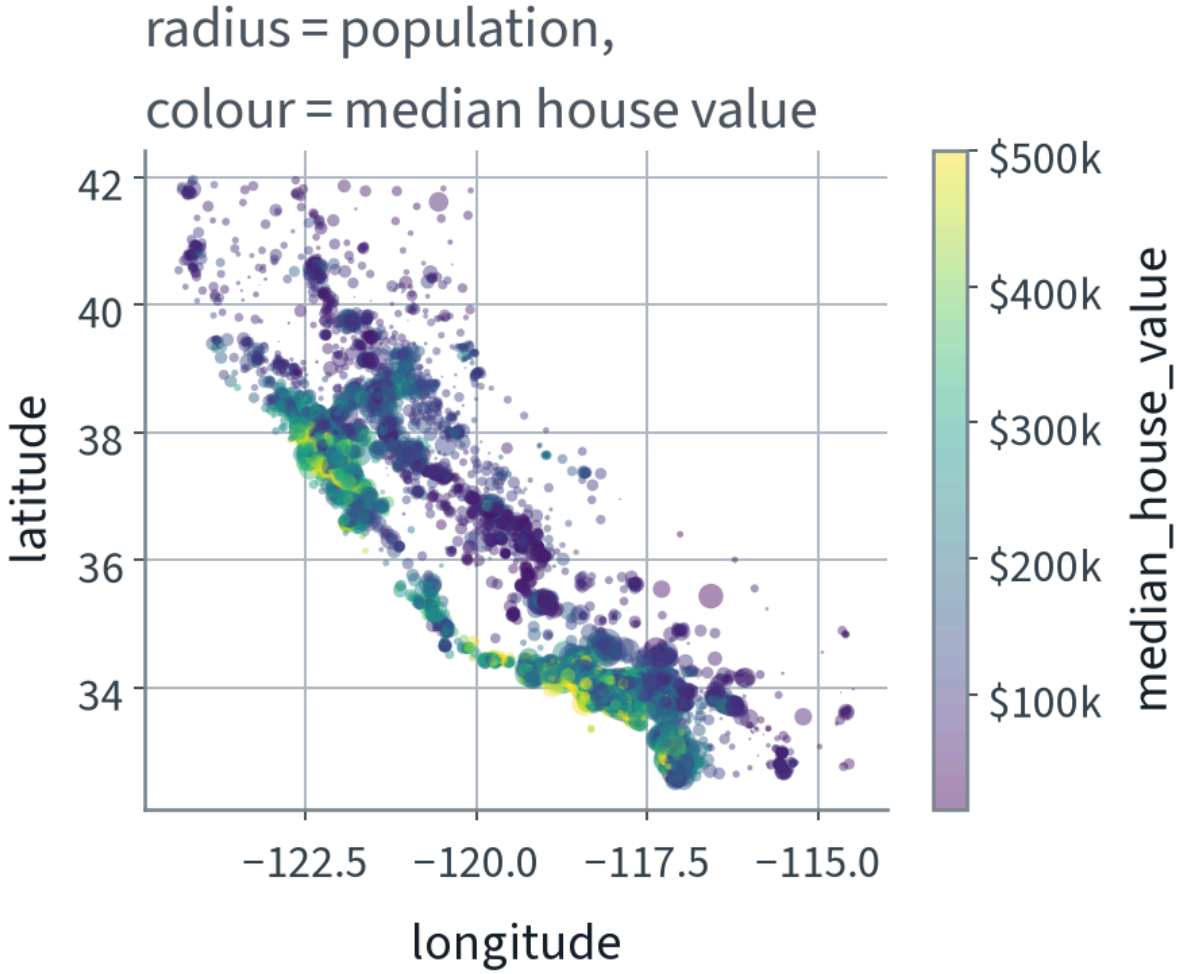
```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
             s=housing["population"] / 100, label="population",
             c="median_house_value", cmap="jet", colorbar=True,
             legend=True, sharex=False, figsize=(10, 7))
plt.show()
```

Radius encodes population, colour encodes price.

Prices are strongly related to **location** and **population density**. A clustering algorithm should be useful here — we will come back to that idea.

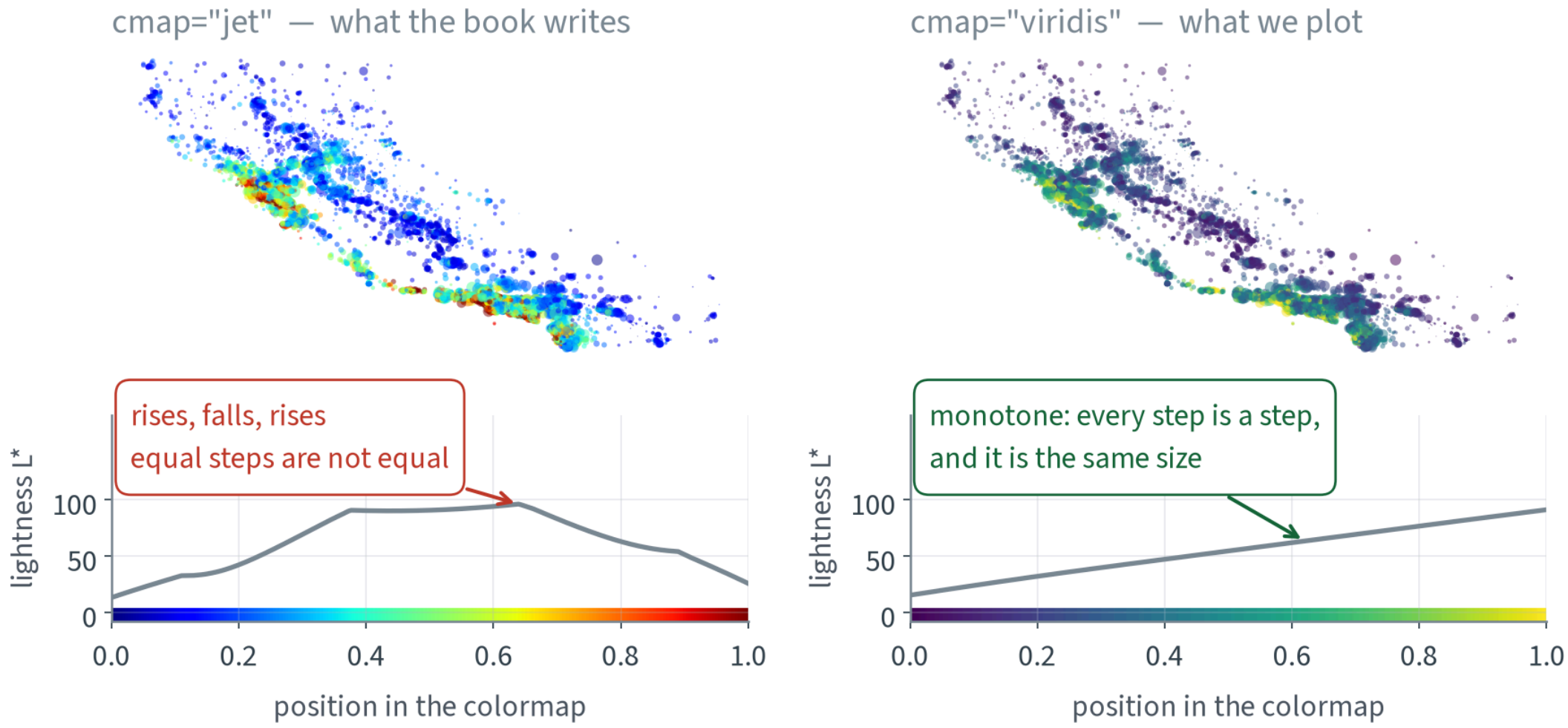
One line in that cell is worth arguing with, and we will — in the next slide but one.

Location and density, against price



Red on the coast, blue inland. The model does not know what “coast” means — but it will learn longitude.

The one line worth arguing with



`cmap="jet"` ran, produced something colourful, and is wrong.

Why, and how you can check it yourself

A colour scale is a claim that equal steps in the data look like equal steps on the screen. The bottom row of that figure is that claim, tested: it plots each colormap's **lightness** against position.

Colormap	Lightness range	Largest reversal	Verdict
✗ `jet`	83.0	**✗ 1.06**	**✗ rises, falls, rises**
`turbo`	78.9	0.87	better, still not monotone
✓ `viridis`	75.9	**0.00 ✓**	**✓ every step is a step**

So `jet` invents bands the data does not have, buries detail in its yellow, and for the 8% of men with a red–green deficiency it reverses rank order outright. Photocopy it and it stops being a scale at all.

> **NOTICE THE SHAPE OF THIS**
>
> The code ran. Nothing errored. The defect was in a default nobody chose deliberately, and it took a measurement to see. **That is every failure in this course, and it is only the second lecture.**

A default nobody chose

FAILURE CONDITION

The code ran and nothing errored. `jet` is matplotlib's default from an older era, and it invents boundaries the data does not have. It took a measurement to see — the same map redrawn on a perceptually uniform scale. A default is a decision somebody made, not the absence of one.

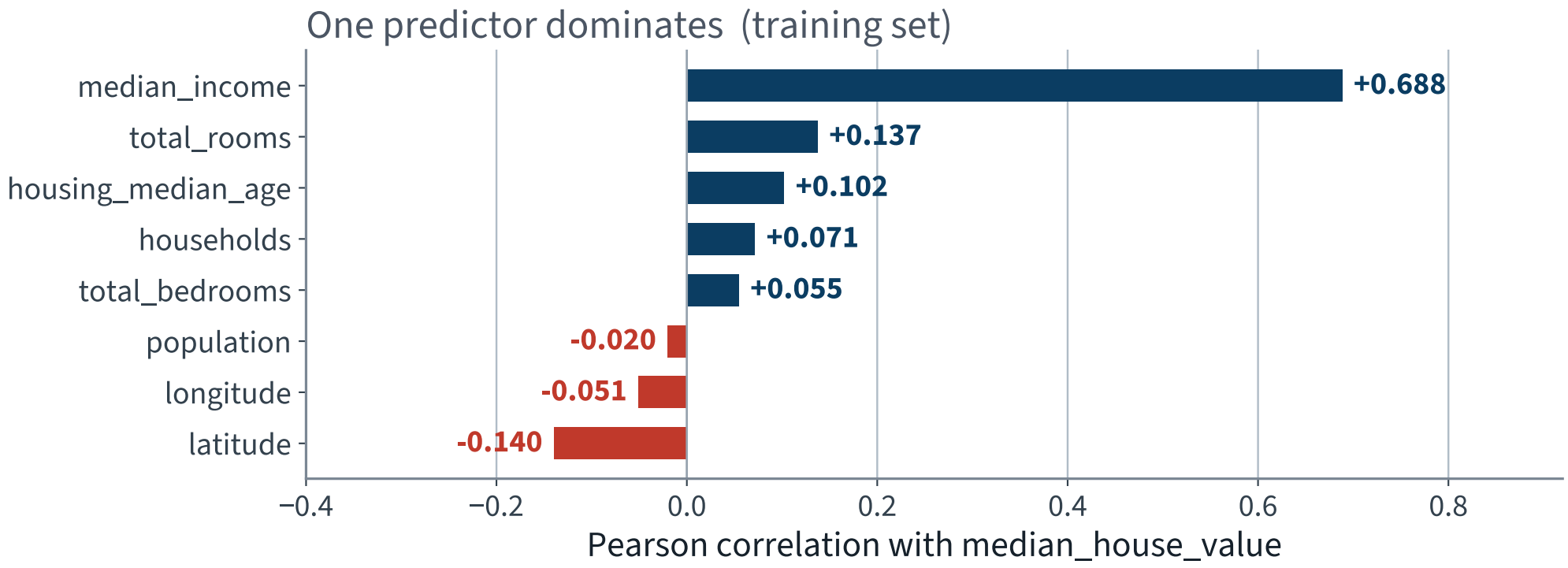
Standard correlation coefficient

```
>>> corr_matrix = housing.corr(numeric_only=True)
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
longitude            -0.050859
latitude             -0.139584
```

Pearson's r ranges from -1 to $+1$. Income dominates.

✗ These are training-set values. Compute them on all 20,640 rows and the third decimal moves — which is why a number is worth nothing without the set it was measured on.

The same column, as a picture



One bar is five times the next. That is the feature we already stratified on — on the experts' word, before we had looked. The measurement says they were right.

What the correlation coefficient cannot see

r measures **linear** correlation only.

TWO TRAPS

A dataset can have $r = 0$ and be completely dependent — any nonlinear relationship. And $r = \pm 1$ says nothing about the *slope*: your height in inches correlates 1.0 with your height in nanometres.

Zoom in on the strongest predictor

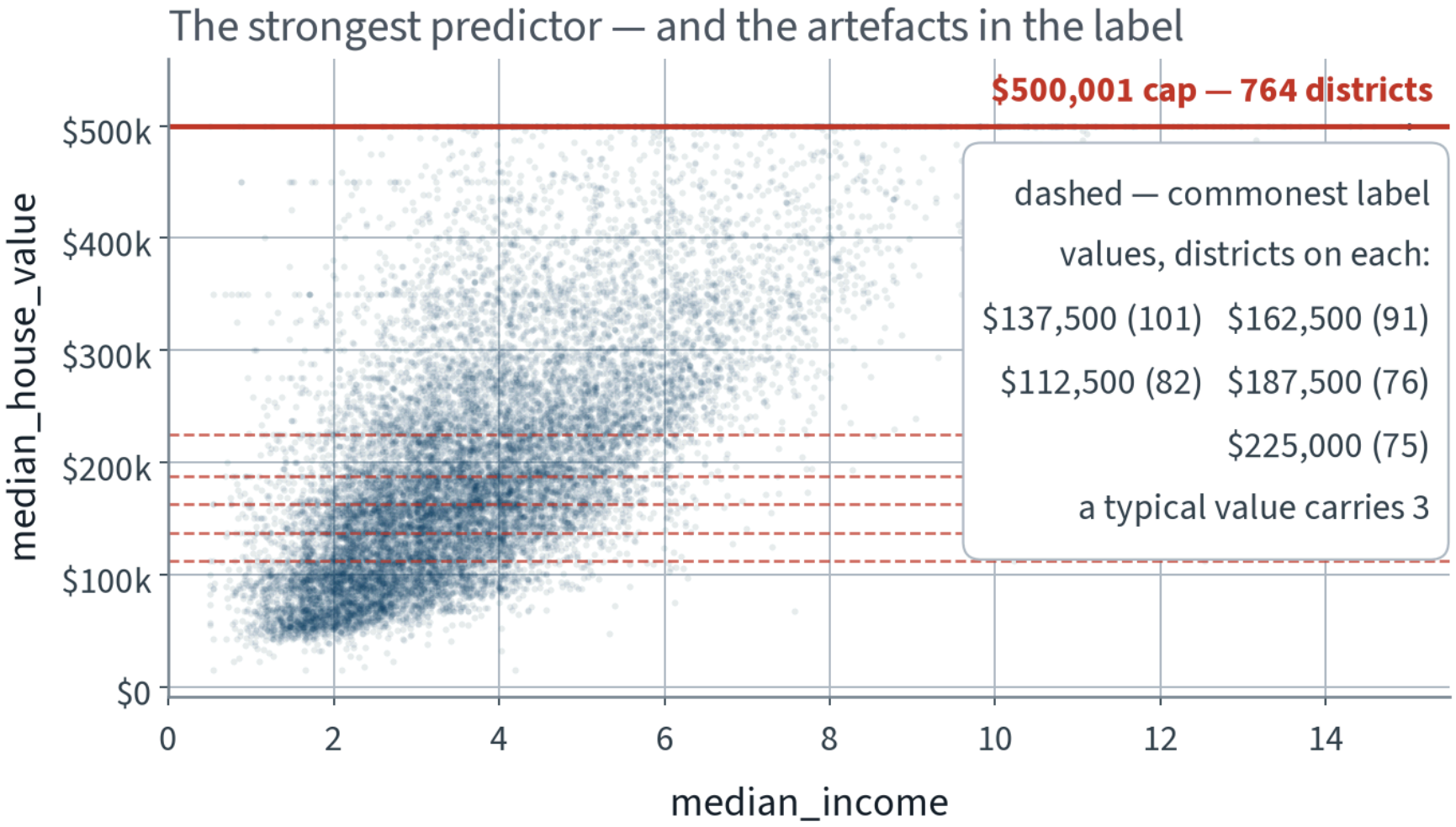
```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1, grid=True)
plt.show()
```

- The upward trend is real, though noisy
- The cap, $500,001, is a clear horizontal line — **✗ 764 districts** in our training split
- There are fainter lines too. **Which ones?** Do not squint at the plot — count.

A horizontal stripe means many districts share one exact price. That is a property of the table, not of California, so it is countable.

> **FAILURE CONDITION — A CAPPED LABEL**
>
> Every district above $500,000 is recorded at the cap, $500,001. A model fits that ceiling perfectly and reports a good error, while being unusable for exactly the districts a client is most likely to ask about. The defect is in the label, so no amount of model choice reaches it.

Income against price — and the lines that are really there



✗ The trend is real. The stripes are not: every one sits on a multiple of $12,500.

What the eye reported, and what the count says

A well-known description of this dataset names three fainter lines. We counted them in our own training split.

Value	Districts	Verdict
$350,000	62	**✓ real — 21× the background**
$450,000	31	marginal
$280,000	**✗ 3**	**✗ indistinguishable from background**

A typical price is shared by **3** districts, so the last one is not a line at all. Meanwhile the five commonest values below the cap — $137,500, $162,500, $112,500, $187,500, $225,000 — go unmentioned, and they are the strongest stripes in the plot.

THE HABIT, NOT THE NUMBERS

Three values were asserted; one survives. Counting took one line. You will inherit descriptions of data for the rest of your career — check them against the data before you build on them.

Combine attributes before modelling

```
housing["rooms_per_house"]  = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"]   = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]
```

Attribute	Correlation with price
median_income	0.688
rooms_per_house	0.144
total_rooms	0.137
bedrooms_ratio	**−0.256 ✓**

`bedrooms_ratio` is far more informative than either raw count.

A warning you will need shortly

COLLINEARITY

When you create combined features, make sure they are not too linearly correlated with existing ones. In particular, **avoid simple weighted sums of existing features.**

Collinearity causes real problems for some models — linear regression above all.

We will see *exactly* why when we derive the normal equation — a derivation this lecture starts, and the next one finishes.

THE MATHEMATICS

Least squares

Géron §4.1, brought forward — before we fit a linear model, we should know what fitting one does

examinable

The question

You called `LinearRegression().fit()`.

What did it actually compute — and when does that computation *fail*?

> **WHY YOU NEED THE ANSWER**
>
> The failure condition is exactly the warning from earlier today about engineering features as weighted sums.

The linear model

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- $\hat{y}$ — the predicted value
- n — the number of features
- x_i — the i-th feature value
- θ_j — the j-th model parameter, including the bias term θ_0

A weighted sum of the input features, plus a constant called the *bias* or *intercept* term.

Vectorised form

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

where $\mathbf{x}$ contains x_0 through x_n with $x_0 \equiv 1$, so the dot product reproduces the bias term.

> In machine learning, vectors are usually **column vectors**. With that convention the prediction is $\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$ — the same quantity, written as a single-cell matrix rather than a scalar.

What “fit” means

Training the model means setting $\boldsymbol{\theta}$ so the model best fits the training set. To do that we need a measure of how badly it fits.

$$\mathrm{MSE}(\mathbf{X}, y, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

MEAN SQUARED ERROR OF A LINEAR HYPOTHESIS

Why minimise MSE and not RMSE?

The **root mean squared error** is the square root of the MSE, $\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$. It is in the units of the target — dollars here — which is why it is the number this course reports. We minimise the MSE instead, because $\sqrt{\cdot}$ is strictly increasing on $[0, \infty)$:

$$\arg\min_{\boldsymbol{\theta}} \sqrt{f(\boldsymbol{\theta})} \;=\; \arg\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$$

Same minimiser, simpler algebra.

> *Learning algorithms often optimise a different function during training than the one used to evaluate the final model — because it is easier to optimise, or because it carries extra terms needed only during training.*

In matrix form

Stack the instances as rows of $\mathbf{X}$ and the labels into y:

$$\mathrm{MSE}(\boldsymbol{\theta}) = \frac{1}{m} \|\mathbf{X}\boldsymbol{\theta} - y\|_2^2$$

We are minimising a squared Euclidean norm. **That is a geometric statement**, and it is the whole content of this thread.

Differentiate and set to zero

$$\nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta}) \;=\; \frac{2}{m}\,\mathbf{X}^{\intercal}(\mathbf{X}\boldsymbol{\theta} - y)$$

Setting the gradient to zero at the minimiser $\hat{\boldsymbol{\theta}}$:

$$\mathbf{X}^{\intercal}\left(\mathbf{X}\hat{\boldsymbol{\theta}} - y\right) \;=\; \mathbf{0}$$

A stationary point. Is it a minimum?

Why that stationary point is *the* minimum

The Hessian of the MSE is

$$\nabla^2_{\boldsymbol{\theta}} \operatorname{MSE} = \frac{2}{m} \mathbf{X}^\intercal \mathbf{X}$$

For any vector $\mathbf{v}$, $\mathbf{v}^\intercal \mathbf{X}^\intercal \mathbf{X} \mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 \geq 0$, so the Hessian is positive semi-definite and the MSE is **convex**.

✓ A convex function has no local minima — only a global one. Any stationary point is it.

This is also why gradient descent, in Lecture 4, has nowhere else to stop. With a suitable learning rate, it reaches this minimum.

The notebook for this lecture

Open Lecture 2 in Colab

- **Runs on free CPU**, and loads the data itself — it stands on its own
- The stratified split and the sampling-bias measurement; the cap and the famous three lines, counted on the training set; the map, the colour-scale test, the correlations and the three ratios
- The mathematics, checked on the training data: the gradient formula against finite differences, and the Hessian's eigenvalues

> **WHAT TO DO WITH IT BEFORE THE NEXT LECTURE**
>
> Run it top to bottom once. Then change one thing — the number of income bands in `pd.cut` — and see what happens to the sampling bias table. Two lines, and it is the whole argument of the stratification section.

Design exercise · in the manner of an ML system-design interview

The task: a rental marketplace’s suggested listing price

> **THE BRIEF**
>
> A rental marketplace — the app where landlords post flats and tenants find them — wants its posting form to **suggest a monthly rent** as the landlord types in the flat, and to badge it “fairly priced” for tenants. The app earns a fee per flat let through it, so it measures how fast listings let and how many landlords post: too high and the flat sits while the landlord tries a rival, too low and the landlord loses money and the app its trust.

- **Given:** the app’s listings — area, rooms, floor, year built, furnished, neighbourhood, asking rent, date posted — and the lease rent of flats let through it
- **Scale:** a few million listings in many cities, tens of thousands new a month; neighbourhoods with thousands, or a few dozen
- **Consumers of the output:** the posting form, the badge tenants see, the landlord’s dashboard

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the centre is today’s: a linear model fitted by least squares, at the end of a pipeline that was split before anyone

Step 1 — Frame it: a rent in euros, and the criterion it is judged by

- Supervised regression, batch: one number per flat, a label on every flat that was let, a history that fits in memory and grows by the month
- Least squares minimises the MSE; what is reported is the RMSE, in euros a month, because it is in the units of the rent and has the same minimiser
- The product team's criterion: within about a tenth of what the flat lets for, or landlords ignore it — a *relative* criterion

> **PITFALL**
>
> An RMSE in euros is dominated by the few penthouses, where a mistake of hundreds is a few per cent; the app's volume is the cheap flats, where the same mistake is a quarter of the rent and the flat sits. The headline can improve while time-to-let, the business's number, gets worse. Write the metric beside the criterion before anything is fitted, and report the error as a share of the rent, by rent band, next to the RMSE.

Step 2 — The label: the rent the flat let for, on the date it let

- One row per flat that was let; the label is the rent in the lease; the date is a column, because rents drift and the model must be able to learn the drift
- The asking rent on every other listing is not a label: a listing is what a landlord hopes for, a lease is what a tenant pays
- `head()`, `info()`, `describe()`, histograms, on the whole history, once: the missing floors and years, the rare heating level, the rents piled on round values

> **PITFALL**
>
> Every listing has an asking rent and only a fraction a lease, so the first attempt trains on asking rents — including the flats that never let because they asked too much. The suggestion then repeats the prices that failed, and time-to-let worsens on the app's own advice. Label with let flats only, the asking rent a feature at most, recent leases, and the months stated.

Step 3 — Split before you look, and measure the split

- Set the test flats aside before one variable is plotted against another, or your eye will choose the features that suit them
- Stratify on what the product team says drives rent — floor-area bands — and *measure* the stratum error of a random split against the stratified one
- Three assertions after: nothing dropped, the same columns on both sides, no listing, and no re-listed flat, on both

> **PITFALL — DATA SNOOPING**
>
> The map of rents, the correlations and the ratios are drawn on the whole history, the features chosen from them, and only then is a test set cut. Nothing errors, and the test error the release rests on is optimistic by an amount the code cannot tell you, because the test flats helped pick the model. Split first, explore a copy of the training flats, and label every exploration number a training-set number.

Step 4 — Features, inside a pipeline

- Ratios where landlords think in ratios: rent is quoted per square metre, area per room carries more than the raw counts, and a ratio of two weak columns can beat both
- The neighbourhood's typical rent per square metre is the strongest single feature, and it is *learned from the labels*
- Every learned step — the median that fills a missing floor, the encoding of the neighbourhood, that typical rent — is a transformer inside one `Pipeline`, fitted on that fit's training flats only

> **PITFALL — LEAKAGE THROUGH A NEIGHBOURHOOD AVERAGE**
>
> The average rent per neighbourhood is computed once, over the whole history, and joined on as a column. Every test flat then carries a feature that contains its own rent, and the test error falls for a reason that will not be there when a landlord posts a new flat, so the suggestion ships worse than reported. Compute the average inside the pipeline, from that fit's training flats, and give a neighbourhood with a few dozen listings the city's figure.

Step 5 — The model: least squares, and when its answer is not unique

- $\hat{y} = \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$, fitted by minimising the squared norm of $\mathbf{X}\boldsymbol{\theta} - y$: a convex cost, so the point where its gradient vanishes is *the* minimum
- Scaling changes nothing for least squares, so the scaler in the pipeline is there for the models that come later, not for this one
- A coefficient per column — euros per square metre, euros for a balcony — is the breakdown the form shows the landlord

PITFALL — COLLINEARITY

The form holds the floor area and a "living area" that is a fixed share of it, and an engineer adds their weighted sum as a size score. The columns are then linearly dependent, the stationarity condition has infinitely many solutions, and the solver returns one whose coefficients mean nothing — while the predictions look fine, so the landlord is shown nonsense as their breakdown. Build ratios, never weighted sums of existing columns, and check the correlations among the features.

Step 6 — Cross-validation, with the fold scores reported

- Split the training flats into k folds; fit the whole pipeline k times, each time scoring the fold it did not see: k RMSEs, their mean *and their spread*
- Every choice — which ratios, which neighbourhoods get the city's figure, recent leases only or all — is made on these folds, never on the test flats
- A difference between two candidates smaller than the fold-to-fold spread is not a difference

> **PITFALL**
>
> Two feature sets are compared by the mean RMSE alone; one is a few euros better and goes into the release, when the folds spread by tens of euros and the order flips from fold to fold. Report the fold scores beside the mean, and choose the simpler candidate when the spread swallows the gap — the form has to explain it anyway.

Step 7 — The test set, once

- Refit the chosen pipeline on all the training flats; `predict` on the test flats; one RMSE, and one error as a share of rent by band
- That number, with the fold spread beside it, is the generalisation error the release note states
- Nothing is fitted on the test flats, and nothing is changed after the number is seen

> **PITFALL**
>
> The test RMSE disappoints, a ratio is added, the test is re-run, and the second number is better — and no longer an estimate of anything, because the test flats have now chosen the model. If the number disappoints, the finding is the number; go back to the folds for the next attempt, and cut a fresh test set from new listings before it is scored.

Step 8 — Release it, and what happens next quarter

- Hand over the fitted pipeline as one object — medians, neighbourhood figures and coefficients together — with the test error by rent band against the criterion, and the loading, split and assertions for the next refit
- Roll out to a share of landlords first, and read time-to-let against the landlords who did not see the suggestion: an A/B test
- Refit on a schedule, on the leases signed since, measuring on them first; the data team is on call for the error by city

> **PITFALL**
>
> Landlords accept the suggestion, so next quarter's asking rents carry the model's own output and a refit on them learns the suggestion, not the market — and a form that nudges every landlord in a street to one number is a question the competition regulator asks. Refit on signed leases, never asking rents; keep the landlords without the suggestion as the control; show a band, not one figure.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Listings → let flats, the lease rent as the label, the date kept → stratified split, measured and asserted → exploration on a training copy → ratios, the neighbourhood figure and the imputer in one pipeline → least squares → k folds, scores reported → the test set once → A/B rollout → refit on signed leases.

Pitfall	Address it by
An RMSE in euros against a relative criterion	the metric beside the criterion; error by rent band
Asking rents, and flats that never let, as labels	let flats only; the asking rent a feature; months stated
Exploring before the split	split first, stratified and measured; a training copy
A neighbourhood average over the whole history	computed inside the pipeline, from training flats
A weighted sum of existing columns	ratios; correlations among the features checked
Candidates crowned on the mean alone	the fold scores and their spread reported
The test set used twice	once; a fresh test set from new listings next time

What we did

1. Set the test set aside before exploring: data snooping is a bias of the analyst, and no library prevents it
2. Stratified on income, and measured it: up to 6.4% off in the income mix for a random split, 0.36% for the stratified one
3. Explored the training set only: a map that needed `alpha`, and a default colour scale whose lightness runs backwards
4. Income dominates the correlations at 0.688; r sees only straight lines; of three asserted stripes in the target, counting kept one
5. A ratio of two weak columns, `bedrooms_ratio`, beat both — and weighted sums of existing columns are to be avoided
6. Wrote the MSE of a linear model, differentiated it, and showed its Hessian $\frac{2}{m}\mathbf{X}^\intercal\mathbf{X}$ is positive semi-definite: the MSE is convex
7. Designed the suggested listing price behind a rental marketplace's posting form in nine steps — a metric written beside a relative criterion, let flats as the label, a stratified split before the exploration, a neighbourhood figure computed inside the pipeline, ratios instead of weighted sums, fold scores reported, the test set used once, and a refit on signed leases behind an A/B rollout — each step named for the pitfall it avoids

Where the derivation stops today

$$\mathbf{X}^\top \mathbf{X}\, \hat{\boldsymbol{\theta}} \;=\; \mathbf{X}^\top y$$

We know what any solution of this is: **the** minimum of the MSE, because the MSE is convex. We have not yet solved it.

- When does it have exactly one solution?
- What is that solution, written in closed form?
- And what happens to it when one feature is a weighted sum of others — the warning from the exploration?

All three are answered by solving for $\hat{\boldsymbol{\theta}}$. That is where the next lecture starts.

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 2

five, in the style of the written paper

Exercises — Lecture 2

1 A colleague reports their model's error on the California housing test set, and says they chose the model by trying several on that same test set and keeping the one with the lowest error. State, in one sentence, what that number is an estimate of — and what it is *not* an estimate of. [4]

2 You are given a stratified split on income category and a naive random split of the same data. A model scored on each test set gives a similar error. Does that mean the stratification was unnecessary? Answer yes or no, with a reason. [4]

3 On the training set, `longitude` has a correlation of $r = -0.05$ with the median house value. A colleague concludes that location does not affect price, and drops the column. State the flaw in one sentence, and name the plot from this lecture that refutes it. [3]

Solutions on Lecture 3's deck.

Exercises — Lecture 2 (continued)

4 The course reports the RMSE of a regression model, yet training minimises the MSE. Explain why the two give the same fitted parameters, and say why the RMSE is the one reported. [3]

5 The Hessian of the MSE of a linear model is $\frac{2}{m}\mathbf{X}^\top\mathbf{X}$. Show that it is positive semi-definite, and state what that guarantees about a point where the gradient vanishes. [5]

Solutions on Lecture 3's deck.

THE FIVE SET AT THE END OF LECTURE 1

Solutions Lecture 1

not part of this lecture

Solutions — Lecture 1

1. The brief says the output feeds a downstream system that expects a price. Name the one thing this fact…

✓ It settles that this is supervised regression, not classification. It leaves the metric open.

- 'A price' fixes the type of the output and nothing about how error is weighed
- Whether being wrong by the same number of dollars on a cheap district matters as much as on an expensive one is a question for the client, not the data — the experts' 30% is a relative criterion, a dollar error is not

2. info() reports that 207 of the 20,640 districts have no value for total_bedrooms. Give the three legitimate…

✓ Drop those districts, drop the whole attribute, or fill the gaps. Filling leaves a decision: the value to fill with.

- Dropping 207 rows, or the column, needs no further choice — each simply has a cost
- "Fill the gaps with a chosen value" leaves the choice open: a constant, the median, the mean, each computed from some set of rows

Solutions — Lecture 1 (2)

3. total_rooms runs from single digits to tens of thousands, median_income from about 0.5 to 15. Name two…

✓ It matters for gradient descent and k-nearest neighbours; it does not for decision trees and ordinary least squares. A tree splits on one feature at a time, so any monotone rescaling gives the same tree.

- Anything that measures a distance, penalises a coefficient or takes a step depends on the units of the features
- Least squares is equivariant under an invertible affine map of the features: the fit is identical and the coefficients absorb the change

4. The median income column is capped at 15. Give one consequence for the model and one for the evaluation, and…

✓ Model: it can never learn what happens above the cap. Evaluation: the test set is capped too, so the error is silent about exactly those districts.

- The cap is in the data before any split, so every held-out set inherits it and no held-out number reveals it
- Raise the evaluation one first: the model can be retrained, but a metric that cannot see the failure will not tell you to

Solutions — Lecture 1 (3)

5. A model's error is low on its training set and high on new districts. Name the condition, and explain…

✓ Overfitting. The training error is measured on cases the model has already seen; the generalisation error is defined on cases it has not.

- A model can fit its training set, noise included, and the training error then rewards it for doing so
- Only data held out from training estimates the error on new cases — which is why the data is split into a training set and a test set

NEXT LECTURE

The normal equation

Solving $\mathbf{X}^\intercal \mathbf{X} \hat{\boldsymbol{\theta}} = \mathbf{X}^\intercal y$ in closed form, and reading it as a projection

When $\mathbf{X}^\intercal \mathbf{X}$ is not invertible — collinear features, too few instances — and the pseudoinverse

Run the Lecture 2 notebook first